Lab 10 — Web Mapping Basics & Linked Dashboards

(HTML + CSS + JavaScript + Plotly)

Open-Source Geovisualization • GEOG 8005 / ESCI 6000 • Spring 2026

Overview

In this lab you will move through two connected stages. Part 1 focuses on the basic building blocks of a webpage: HTML structure, CSS styling, and JavaScript interactions. Part 2 uses a provided Jupyter notebook and a reusable HTML template to generate a small linked Plotly dashboard and export it as a standalone webpage.

The goal is not only to run the starter files, but also to read the code, understand which lines control key design choices, and improve the final webpage and dashboard so they look intentional and easier to use.

What you will produce (portfolio-friendly)

- One edited basic webpage that demonstrates HTML structure, CSS styling, and JavaScript interaction.
- One final linked dashboard HTML file with a map, bar chart, scatter plot, and shared data table.
- A small set of screenshots showing webpage edits, JavaScript behavior, and linked dashboard interaction.
- Short written answers to the code-reading and design-control questions.

Prerequisites

- A working conda environment (for example, geoviz) with pandas and plotly installed.
- Jupyter Notebook, JupyterLab, VS Code, or another editor that can run notebooks and edit HTML, CSS, JavaScript, and Python files.
- A modern browser such as Chrome, Edge, or Firefox for previewing HTML output.
- Recommended: basic familiarity with reading simple Python functions and editing text files.

```
# Example environment check
python -c "import pandas, plotly; print('OK')"
```

Files and data for this lab

Use the provided sample file data in the Jupyter notebook It contains a small city-level table used to build the dashboard in Part 2.

- city_id: shared identifier used to link chart marks and table rows
- city / city_label: city names used for display
- lat / lon: point coordinates for the map
- population, health_index, green_space, avg_temp: simple variables used to control symbol size, color, labels, and hover content

Files provided with this package:

- starter_files/part1_basic_webpage/index.html
- starter_files/part1_basic_webpage/styles.css
- starter_files/part1_basic_webpage/script.js
- starter_files/part2_linked_dashboard/demo_interactive_final.ipynb
- starter_files/part2_linked_dashboard/dashboard_template_new.html

Note: as in Lab 7, the starter files are meant to be improved. Do not leave the webpage or the dashboard exactly as the default version if readability, hierarchy, or interaction cues can be improved.

Submission (screenshots + files)

Submit screenshots and source files containing the following evidence:

- Successful imports or environment check.
- Part 1: screenshot of the basic webpage after HTML and CSS edits.
- Part 1: screenshot showing the JavaScript interaction in action.
- Part 2: screenshot of the final linked dashboard layout.
- Part 2: screenshot showing one linked selection across multiple views.
- Part 2: screenshots showing the creation of the CSS and JS files, and the HTML will not have a related code block, but still works properly.
- Source files: index.html, styles.css, script.js, demo_interactive_final.ipynb, dashboard_template_new.html, and your final lab10_dashboard_output.html.
- Short written answers to the check-your-understanding questions.

Part 1 — Web page basics

1.1 Project folders

Keep the basic webpage files together so the HTML, CSS, and JavaScript are easy to preview and debug.

1.2 Open the starter page and identify HTML structure

Open starter_files/part1_basic_webpage/index.html in your editor. Identify the doctype, html, head, body, title, link, script, header, main, section, figure, table, and button elements. Preview the page in a browser and compare the visible content with the HTML structure.

1.3 Basic HTML tasks

- Replace the placeholder introduction text with a short topic of your choice.
- Add one more bullet item and at least one more table row.
- Change the figure caption and, if you prefer, replace the image.
- Add one additional hyperlink to a relevant site.
- Make sure the page title in the browser tab is meaningful.

1.4 CSS tasks

- Open styles.css and identify where the page background, hero section, cards, table, button, and responsive layout are controlled.
- Modify at least three design features such as colors, spacing, border radius, font size, or hover state.
- Improve the visual hierarchy so the title, section headings, and body text are easier to scan.
- Check that the page still works reasonably on a narrow browser window.

1.5 JavaScript tasks

- Open script.js and identify the event listener, class toggle, and text updates.
- Demonstrate the show/hide interaction in the browser.
- Add one small additional interaction of your own, such as changing button color after clicking, swapping the caption text, or adding a second toggle.

Check your understanding (Part 1)

- In HTML, what is the difference between an element and an attribute?
- Why do we usually separate content (HTML), presentation (CSS), and behavior (JavaScript)?
- Which lines in your CSS control the page background, heading appearance, and button styling?
- What does the JavaScript event listener do, and what visible change does it trigger on the page?
- Why is it useful to preview your webpage in both the editor and the browser while you work?

Part 2 — Python-generated linked dashboard

2.1 Inspect notebook starter

Open starter_files/part2_linked_dashboard/demo_interactive_final.ipynb and read the first two cells carefully.

2.2 Run the Part 2 notebook

- Run the notebook from starter_files/part2_linked_dashboard/ so Path.cwd() points to that folder.
- Confirm that the notebook finds dashboard_template_new.html, then run all cells.
- Make sure the notebook writes lab10_dashboard_output.html successfully.

- This starter is notebook-safe and uses Path.cwd(); do not switch to __file__ inside Jupyter.

```
cd starter_files/part2_linked_dashboard
# then open and run demo_interactive_final.ipynb
```

2.3 Code-reading and design-control tasks

- In make_map, identify which lines control marker color, marker size, map style, map center, and zoom.
- In make_bar, identify which lines control ordering, bar color, text labels, and x-axis styling.
- In make_scatter, identify which lines control marker size, marker color, and text placement.
- In make_table, identify how city_id is carried into the HTML table rows.
- In make_js_chart_data, explain why the notebook pre-embeds a JavaScript data object for the interaction layer.
- Modify at least three design choices so the final dashboard looks more intentional than the default starter version.

Dashboard tuning (what to modify):

- Map symbols: change the size scaling, color logic, hover design, or map style.
- Bar chart readability: adjust sort order, label placement, margins, or axis styling.
- Scatter plot styling: change color palette, marker outlines, text position, or background grid treatment.
- Table readability: improve spacing, selected-row emphasis, or hover behavior.
- File simplification: instead of embedding the styles and JavaScript in the the html, create two separate files: styles.css and scripts.js to store the styles and JavaScript code.
- Overall layout: improve visual hierarchy through header text, badge text, panel titles, or spacing.

2.4 Template assembly tasks

- Open dashboard_template_new.html and identify the placeholder tokens such as {{MAP_CHART}}, {{TABLE_HTML}}, and {{JS_CHART_DATA}}.
- Explain how the notebook replaces these tokens with generated HTML strings and an embedded JavaScript state object.
- Identify where the template controls the hero header, grid, panel styling, table styling, and selection/highlight behavior.
- In this version the CSS and JavaScript are embedded directly in the template instead of split across separate files.

2.5 Linked interaction tasks

- Trace how window.CHART_DATA, extractCityId, selectCity, and clearSelection are used to connect map clicks, bar clicks, scatter clicks, and table row clicks.
- Demonstrate at least one click-based linked selection and one clearing action (Esc or double-click).

- Change one interaction-related visual cue such as highlight color, outline width, selected-row styling, or panel-active styling.

Check your understanding (Part 2)

- Why is city_id useful as a shared key across the map, charts, and table?
- Which function in the notebook is responsible for replacing the placeholder tokens in the HTML template?
- Why does this version embed window.CHART_DATA before the interaction code runs?
- What parts of the final dashboard are primarily controlled by Python, which by HTML/CSS, and which by JavaScript?
- In the interaction code, what happens when a user clicks a point or bar, and how is the selection cleared?

Completion checklist

- Part 1 webpage shows meaningful HTML edits rather than only text replacement.
- CSS modifications clearly improve hierarchy, spacing, and readability.
- JavaScript interaction is demonstrated and explained.
- Part 2 notebook runs successfully and produces an output HTML file.
- At least three dashboard design choices are modified intentionally.
- Screenshot showing the creation of the CSS and JS files and the html will not have related code block but still works properly.
- Linked interaction is demonstrated with screenshot evidence.
- Short written answers are included for the check-your-understanding questions.